

Computer Vision – Assignment 5

Vasiliki Zerva

Segmentation using background subtraction

```
8  #Loading of the video
9  video_path = 'static_camera.webm'
10 cap = cv2.VideoCapture(video_path)
11
12 #It reads the first frame to use it
13 ret, frame = cap.read()
14 if not ret:
15     raise Exception("Can't read the video.")
16
17 gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
18 height, width = gray_frame.shape
19 num_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

The first necessary thing is the loading of the video and the read of the first frame.

```
21  #Accumulated Weighted Image
22  accum_weight = 0.01
23  accum_background = np.float32(gray_frame)
24
25  #MOG2 Background Subtractor
26  mog2 = cv2.createBackgroundSubtractorMOG2()
27
28  #Custom Moving Average Background
29  custom_background = np.zeros_like(gray_frame, dtype=np.float32)
30
31  #Store foregrounds for the chosen frame
32  chosen_frame_index = num_frames // 2
33  foregrounds = {}
```

The set of the Accumulated Weighted Image, the backgrounds and the foregrounds.

```

35 #Process video
36 cap.set(cv2.CAP_PROP_POS_FRAMES, 0) # Restart video
37
38 for i in range(num_frames):
39     ret, frame = cap.read()
40     if not ret:
41         break
42
43     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
44
45     #Accumulate Weighted
46     cv2.accumulateWeighted(gray, accum_background, accum_weight)
47     fg1 = cv2.absdiff(gray, cv2.convertScaleAbs(accum_background))
48
49     #MOG2
50     fg2 = mog2.apply(frame)
51
52     #Custom Moving Average
53     custom_background = (custom_background * i + gray) / (i + 1)
54     fg3 = cv2.absdiff(gray, cv2.convertScaleAbs(custom_background))
55
56     #Save foregrounds at the chosen frame
57     if i == chosen_frame_index:
58         foregrounds['frame'] = frame
59         foregrounds['fg1'] = fg1
60         foregrounds['fg2'] = fg2
61         foregrounds['fg3'] = fg3

```

During the process of the video, we set the functions for the Accumulated Weight, the MOG2 and the Moving Average. Also, it is really important to set and save the foregrounds.

```

63 #Final background models
64 final_bg1 = cv2.convertScaleAbs(accum_background)
65 final_bg2 = mog2.getBackgroundImage()
66 final_bg3 = cv2.convertScaleAbs(custom_background)
67
68 cap.release()
69
70 #Visualization
71 def show_image(title, img, cmap='gray'):
72     plt.figure(figsize=(4, 4))
73     plt.title(title)
74     plt.imshow(img, cmap=cmap)
75     plt.axis('off')
76
77 #Background
78 show_image("Background - Accumulate Weighted", final_bg1)
79 show_image("Background - MOG2", final_bg2)
80 show_image("Background - Moving Average", final_bg3)
81
82 #Original Frame and Foreground masks
83 show_image("Original Frame", cv2.cvtColor(frame, cv2.COLOR_BGR2RGB), cmap=None)
84 show_image("Foreground - Accumulate Weighted", foregrounds['fg1'])
85 show_image("Foreground - MOG2", foregrounds['fg2'])
86 show_image("Foreground - Moving Average", foregrounds['fg3'])
87
88 plt.show()

```

The backgrounds are set and the results are printed.

Results

Original Frame



Foreground - AccumulateWeighted



Foreground - MOG2



Foreground - Moving Average



Background - AccumulateWeighted



Background - MOG2



Background - Moving Average

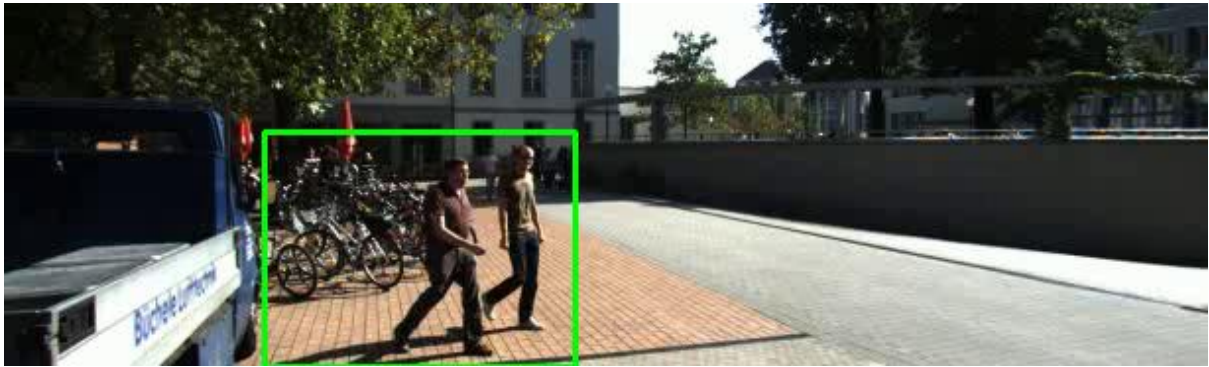


These are the results that we get for the Foregrounds and the Backgrounds from the implementation of the Accumulated Weighted, MOG2 and Moving Average.

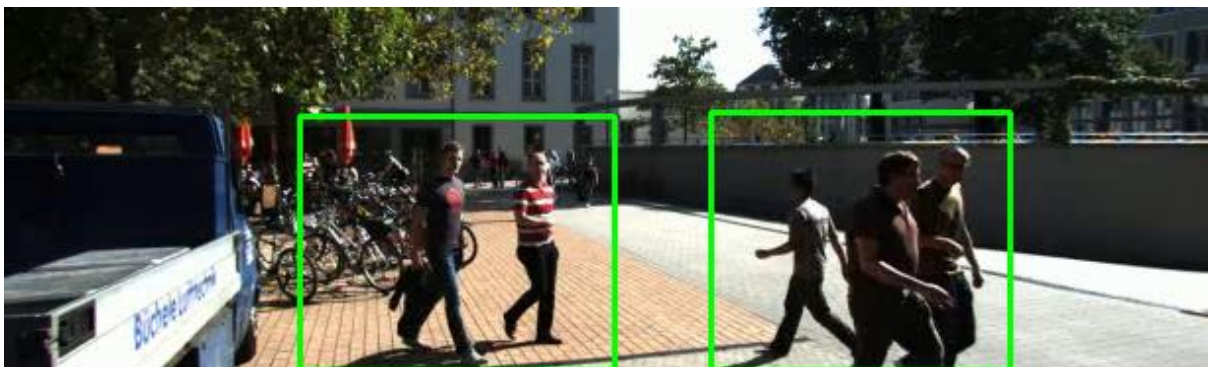
Dense Optical Flow

Results

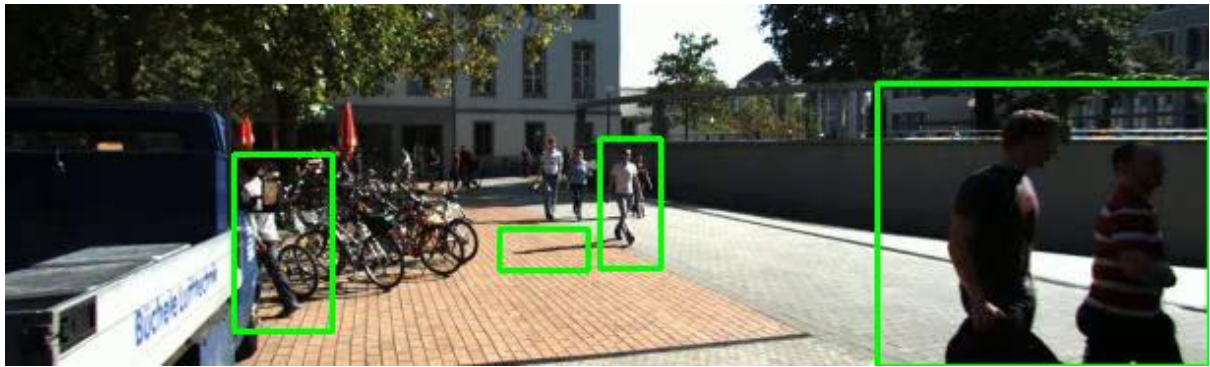
Frame 2



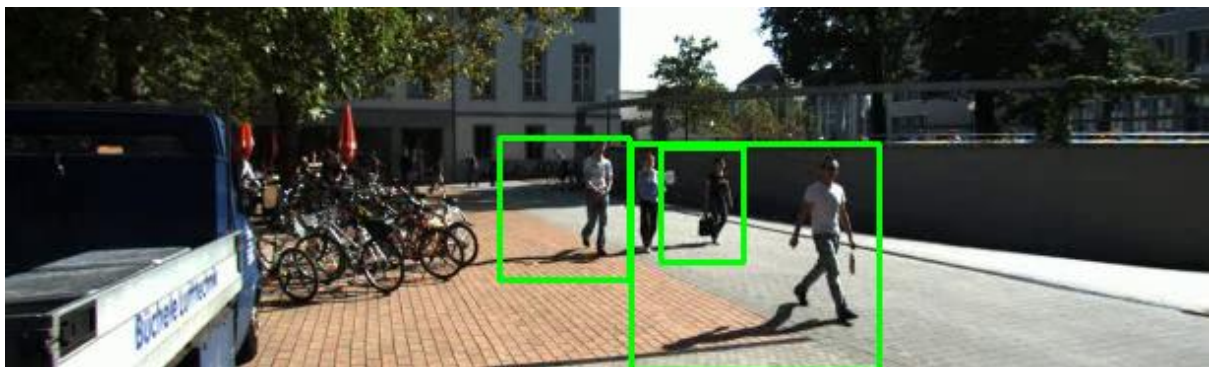
Frame 32



Frame 62



Frame 92



Frame 122

